

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.14

VECTOR & GEOGRAPHY FIELDS

 Copy as Markdown

Vector & geography fields

Serverpod supports two families of specialized field types backed by PostgreSQL extensions: vector fields for high-dimensional embeddings (via pgvector) and geography fields for geospatial data (via PostGIS). Column types, JSON encodings, and SQLite storage are listed in [Field types](#).

Vector fields

Vector types are used for storing high-dimensional vectors, which are especially useful for similarity search operations.

When specifying vector types, the dimension is required between parentheses (for example, `Vector(1536)`). Common dimensions include:

- 1536 (OpenAI embeddings)
- 768 (many sentence transformers)
- 384 (smaller models)

All vector types support specialized distance operations for similarity search and filtering. See the [Vector distance operators](#) section for details.

To ensure optimal performance with vector similarity searches, consider creating specialized vector indexes on your vector fields. See the [Vector indexes](#) section for more details.

INFO

Vector fields require a PostgreSQL database with the `pgvector` extension installed. New projects have it. The embedded PostgreSQL bundles pgvector 0.8.3, and the ghcr.io/serverpod/postgres Docker image ships with it as well. SQLite does not support vector queries. To add the extension to an older Docker project or an external Postgres, see the [Upgrade to pgvector](#) guide.



Ask AI

Vector

The `Vector` type stores full-precision floating-point vectors for general-purpose embeddings.

```
class: Document
table: document
fields:
  ### The category of the document (e.g., article, tutorial).
  category: String

  ### The contents of the document.
  content: String

  ### A vector field for storing document embeddings
  embedding: Vector(1536)
```

HalfVector

The `HalfVector` type uses half-precision (16-bit) floating-point numbers, providing memory savings with acceptable precision loss for several applications.

```
class: Document
table: document
fields:
  content: String
  ### Half-precision embedding for memory efficiency
  embedding: HalfVector(1536)
```

SparseVector

The `SparseVector` type efficiently stores sparse vectors where most values are zero, which is ideal for high-dimensional data with few non-zero elements.

```
class: Document
table: document
fields:
  content: String
  ### Sparse vector for keyword-based embeddings
  keywords: SparseVector(10000)
```

Bit

The `Bit` type stores binary vectors where each element is 0 or 1, offering maximum memory efficiency for binary embeddings.

```
class: Document
table: document
fields:
  content: String
```

```
### Binary vector for semantic hashing
hash: Bit(256)
```

Reading vector values in Dart

All four vector types implement `Iterable` and support zero-based indexed access with `[]`. Values read from the database can therefore be inspected directly, without converting them to a list first.

```
var embedding = Vector([0.2, 0.4, 0.6]);

print(embedding.length); // 3
print(embedding[1]); // 0.4

for (var value in embedding) {
  print(value);
}

var hasLargeValue = embedding.any((value) ⇒ value > 0.5);
var values = embedding.toList();
```

The `Vector`, `HalfVector`, and `SparseVector` types iterate over `double` values, while `Bit` iterates over `bool` values.

```
var flags = Bit([true, false, true]);

print(flags[1]); // false
print(flags.where((value) ⇒ value).length); // 2
```

A `SparseVector` iterates over every dimension, including the zero dimensions it does not store internally. Indexed access and `toList()` return `0.0` for those dimensions.

```
var sparse = SparseVector([1.5, 0.0, 0.0, 2.5]);

print(sparse.length); // 4
print(sparse[1]); // 0.0
print(sparse.toList()); // [1.5, 0.0, 0.0, 2.5]
```

Geography fields

Geography types are used for storing geospatial data on the surface of the Earth. They are stored as PostGIS geography columns in PostgreSQL using the WGS 84 coordinate system (SRID 4326), which is the standard used by GPS. The SRID is fixed to `4326`, available in Dart as `Geography.defaultSrid`; configuring a different SRID per column is not yet supported.

Like other fields, geography fields can be made optional by appending `?` to the type, for example `location: GeographyPoint?`.

All geography types support spatial filter operations such as proximity search, intersection, containment, and distance-based ordering. See the [Geography operators](#) section for details.

To ensure optimal performance with spatial queries, consider creating a spatial index on your geography fields. See the [Geography indexes](#) section for more details.

INFO

Geography fields require the PostGIS PostgreSQL extension. New projects have it. The embedded PostgreSQL bundles PostGIS 3.5.4, and the `ghcr.io/serverpod/postgres:16` Docker image ships with it as well. To set it up on an older Docker project or an external Postgres, see the [Upgrading to PostGIS support guide](#).

WARNING

Spatial operations are only supported on PostgreSQL. On SQLite, geography values are stored as text in EWKT format and round-trip through CRUD operations, but spatial query operations are not supported and throw at query time.

GeographyPoint

The `GeographyPoint` type stores a single geographic location defined by longitude and latitude.

```
class: Store
table: store
fields:
  name: String
  location: GeographyPoint
  address: String?
```

GeographyLineString

The `GeographyLineString` type stores an ordered sequence of points forming a path or route.

```
class: DeliveryRoute
table: delivery_route
fields:
  name: String
  path: GeographyLineString
  description: String?
```

GeographyPolygon

The `GeographyPolygon` type stores a closed region defined by an exterior ring and optional interior holes.

```
class: DeliveryZone
table: delivery_zone
fields:
  name: String
  boundary: GeographyPolygon
  description: String?
```

GeographyGeometryCollection

The `GeographyGeometryCollection` type stores a collection of mixed geography types (points, lines, and polygons) as a single field.

```
class: Region
table: region
fields:
  name: String
  features: GeographyGeometryCollection
```